UJIAN TENGAH SEMESTER PEMROGRAMAN PERANGKAT LUNAK BERORIENTASI SERVICE

Disusun Oleh:

Raymond Divian Nathaniel 2410511079

PROGRAM STUDI S-1 INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA
2026

1. Latar Belakang dan Tujuan

Dalam era digital saat ini, kemudahan akses dan efisiensi menjadi kunci dalam berbagai layanan, termasuk pemesanan fasilitas olahraga. Banyak penyedia lapangan olahraga masih mengandalkan sistem manual atau platform yang kurang terintegrasi, menyebabkan kesulitan bagi pengguna dalam mencari, membooking, dan membayar lapangan. Selain itu, pengembangan aplikasi monolitik seringkali menghadapi tantangan dalam skalabilitas, pemeliharaan, dan penanganan kegagalan.

Proyek ini muncul sebagai solusi untuk mengatasi masalah tersebut dengan membangun sistem booking lapangan olahraga yang modern, efisien, dan mudah diakses. Fokus utama adalah pada penggunaan arsitektur microservices untuk memastikan skalabilitas, ketahanan, dan kemudahan pengembangan di masa depan.

Tujuan

Tujuan utama dari proyek ini adalah:

1. Menyediakan Platform Booking Online: Memungkinkan pengguna untuk dengan mudah mencari, melihat ketersediaan, dan membooking lapangan olahraga secara online.
2. Meningkatkan Efisiensi Operasional: Mengotomatisasi proses booking dan pembayaran, mengurangi beban kerja administrasi bagi penyedia lapangan.
3. Membangun Sistem yang Skalabel dan Resilien: Menggunakan arsitektur microservices untuk memastikan sistem dapat tumbuh dan beradaptasi dengan kebutuhan di masa depan, serta tetap berfungsi meskipun ada kegagalan di salah satu komponen.
4. Menerapkan Praktik Keamanan Terbaik: Melindungi data pengguna dan transaksi melalui mekanisme autentikasi dan otorisasi yang kuat.
5. Memfasilitasi Integrasi dan Pengembangan: Memisahkan fungsionalitas menjadi layanan-layanan kecil yang independen, memudahkan pengembangan, deployment, dan pemeliharaan oleh tim yang berbeda.

2. Ruang Lingkup Proyek

Proyek ini mencakup pengembangan API backend untuk sistem booking lapangan olahraga. Fungsionalitas utama yang dicakup meliputi:

- Manajemen Pengguna (Auth Service): Registrasi, login, autentikasi (JWT), refresh token, logout, dan manajemen profil pengguna.
- Manajemen Lapangan (Field Service): Pencarian dan daftar lapangan, detail lapangan, ketersediaan slot waktu berdasarkan tanggal, serta manajemen lapangan oleh pemilik (opsional).

- Manajemen Booking dan Pembayaran (Booking Service): Pembuatan booking, pembayaran DP dan pelunasan, pembatalan booking, dan riwayat booking pengguna.
- API Gateway: Sebagai titik masuk tunggal untuk semua permintaan klien, menangani routing, autentikasi awal, rate limiting, dan penanganan kesalahan.

Proyek ini berfokus pada pengembangan API backend dan tidak mencakup pengembangan antarmuka pengguna (frontend) secara spesifik, meskipun API dirancang untuk dapat diintegrasikan dengan berbagai jenis klien (web, mobile).

3. Arsitektur Sistem

Sistem ini dirancang menggunakan pendekatan Microservices Architecture yang dikoordinasikan oleh sebuah API Gateway.

3.1. Arsitektur Microservices

Microservices adalah pendekatan arsitektur di mana aplikasi dibangun sebagai kumpulan layanan-layanan kecil yang independen, dapat di-deploy secara terpisah, dan berkomunikasi melalui API yang terdefinisi dengan baik.

Keuntungan dalam Proyek Ini:

- Setiap layanan dapat diskalakan secara terpisah sesuai kebutuhannya. Misalnya, Auth Service dapat diskalakan lebih tinggi saat banyak permintaan login/register tanpa mempengaruhi Field Service.
- Kegagalan pada satu layanan (misalnya, Field Service) tidak akan menyebabkan seluruh sistem down, karena layanan lain tetap beroperasi.
- Tim yang berbeda dapat bekerja pada layanan yang berbeda secara bersamaan tanpa saling bergantung terlalu banyak.
- Memungkinkan penggunaan teknologi yang berbeda untuk setiap layanan (misalnya, Node.js untuk Auth/Field, Laravel untuk Booking).

Penerapan dalam Proyek: Proyek ini membagi fungsionalitas menjadi tiga layanan utama:

1. Auth Service (Node.js): Bertanggung jawab atas semua hal terkait autentikasi dan otorisasi pengguna.
2. Field Service (Node.js): Bertanggung jawab atas manajemen data lapangan olahraga dan slot waktu.
3. Booking Service (Laravel 11): Bertanggung jawab atas proses booking, pembayaran, dan ulasan.

3.2. Penggunaan JWT (JSON Web Tokens)

JWT digunakan sebagai mekanisme utama untuk autentikasi dan otorisasi dalam sistem.

Cara Kerja:

1. Login: Ketika pengguna berhasil login melalui /auth/login di Auth Service, server akan membuat dua jenis token:
    - Access Token: Sebuah JWT yang berisi informasi dasar pengguna (ID, email, nama) dan memiliki masa berlaku singkat (misalnya, 15 menit). Token ini digunakan untuk mengautentikasi setiap permintaan ke endpoint yang dilindungi.
    - Refresh Token: Sebuah JWT yang memiliki masa berlaku lebih panjang (misalnya, 7 hari). Token ini digunakan untuk mendapatkan Access Token baru setelah Access Token lama kadaluarsa, tanpa perlu login ulang. Refresh token disimpan di database dan dapat dicabut (revoke) saat logout.
2. Akses ke Protected Routes: Setiap kali klien ingin mengakses endpoint yang memerlukan autentikasi (misalnya, /auth/profile, /bookings), klien harus menyertakan Access Token di header Authorization: Bearer <access_token>.
3. Verifikasi Token: API Gateway akan mencegat permintaan ini, memverifikasi Access Token. Jika valid, permintaan diteruskan ke layanan yang sesuai. Jika tidak valid atau kadaluarsa, Gateway akan menolak permintaan atau mengarahkan ke proses refresh token.
4. Refresh Token Flow: Ketika Access Token kadaluarsa, klien dapat mengirim Refresh Token ke endpoint /auth/refresh di Auth Service untuk mendapatkan Access Token dan Refresh Token baru. Refresh token lama akan dicabut.
5. Logout: Klien mengirim Refresh Token ke endpoint /auth/logout di Auth Service. Refresh Token tersebut akan dicabut dari database, sehingga tidak dapat lagi digunakan untuk mendapatkan Access Token baru.

3.3. API Gateway

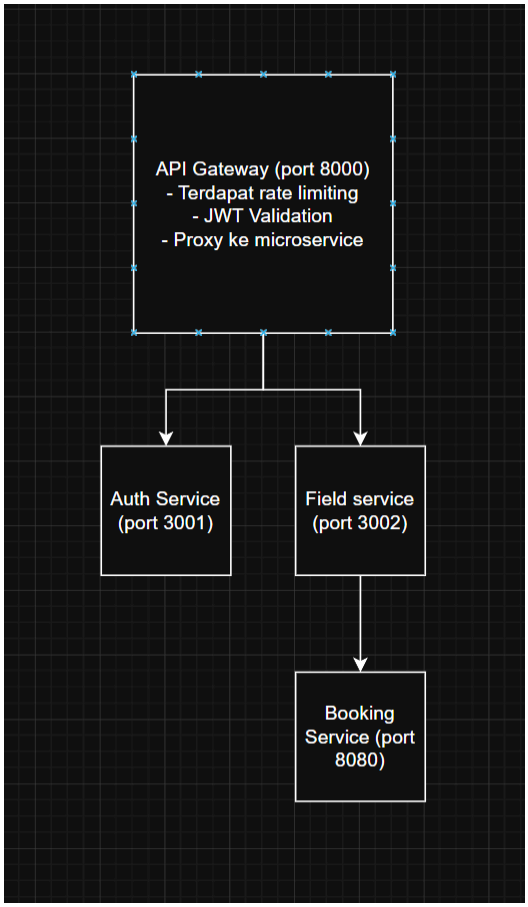
API Gateway bertindak sebagai *single entry point* untuk semua permintaan klien ke sistem microservices.

Fungsi :

- Routing Permintaan: Meneruskan permintaan masuk ke layanan microservices yang sesuai berdasarkan path URL (misalnya, /auth/* ke Auth Service, /fields/* ke Field Service, /bookings/* ke Booking Service).
- Autentikasi dan Otorisasi: Mencegat permintaan, memverifikasi JWT Access Token. Jika token valid, informasi pengguna (dari payload JWT) dapat disuntikkan ke permintaan sebelum diteruskan ke layanan backend. Ini mengurangi beban autentikasi di setiap microservice.

- Rate Limiting: Mencegah penyalahgunaan API dengan membatasi jumlah permintaan yang dapat dibuat oleh klien dalam periode waktu tertentu.
- Penanganan Kesalahan Global: Menyediakan respons kesalahan yang konsisten untuk seluruh sistem.
- Load Balancing (Implisit): Meskipun tidak ada load balancer eksternal, Gateway dapat dikonfigurasi untuk mendistribusikan permintaan ke beberapa instance layanan jika ada.
- Optimasi Koneksi: Mengelola koneksi HTTP ke layanan backend dengan optimasi seperti keep-alive dan socket management untuk mencegah socket hang up.

4. Arsitektur sistem yang dipakai



Databases:

- auth_db (MySQL) - User & tokens
- field_db (MySQL) - Lapangan & slots
- booking_db (MySQL) - Bookings & pembayaran

Penjelasan:

- API Gateway: Dibangun dengan Node.js/Express.js. Menerima semua permintaan, melakukan validasi JWT, rate limiting, dan kemudian mem-proxy permintaan ke microservices yang sesuai.
- Auth Service: Dibangun dengan Node.js/Express.js. Mengelola data pengguna dan proses autentikasi/otorisasi. Berinteraksi dengan auth_db.
- Field Service: Dibangun dengan Node.js/Express.js. Mengelola data lapangan dan slot waktu. Berinteraksi dengan field_db.
- Booking Service: Dibangun dengan Laravel 11 (PHP). Mengelola proses booking, pembayaran, dan ulasan. Berinteraksi dengan booking_db. Layanan ini juga melakukan panggilan inter-service ke Field Service untuk menandai slot sebagai tidak tersedia atau tersedia kembali.

5. Alur Komunikasi Sistem

1. Semua permintaan HTTP/HTTPS pertama kali tiba di API Gateway (port 8000).
2. Gateway ke Auth Service:
    - Untuk endpoint /auth/* (register, login, refresh, logout, profile), Gateway akan mem-proxy permintaan ke Auth Service (port 3001).
    - Untuk endpoint lain yang dilindungi (misalnya, /bookings, /fields yang memerlukan autentikasi), Gateway akan memvalidasi JWT Access Token. Jika valid, Gateway akan menyuntikkan informasi pengguna dari token ke objek permintaan (misalnya, req.user) sebelum mem-proxy ke layanan tujuan.
3. Gateway ke Field Service: Untuk endpoint /fields/*, Gateway akan mem-proxy permintaan ke Field Service (port 3002).
4. Gateway ke Booking Service: Untuk endpoint /bookings/*, Gateway akan mem-proxy permintaan ke Booking Service (port 8080).
5. Inter-Service Communication (Booking Service ke Field Service):
    - Ketika sebuah booking baru dibuat di Booking Service, Booking Service akan memanggil API Field Service (misalnya, POST /fields/{slot_id}/mark-unavailable) untuk menandai slot waktu yang dipesan sebagai tidak tersedia.
    - Ketika booking dibatalkan atau dihapus, Booking Service akan memanggil API Field Service (misalnya, POST /fields/{slot_id}/mark-available) untuk mengembalikan ketersediaan slot.

    - Booking Service juga dapat memanggil Field Service untuk mendapatkan detail lapangan (GET /fields/{id}) saat membuat booking.

6. Desain Database (Setiap Service Memakai Database Sendiri)

Setiap microservice memiliki database-nya sendiri, yang merupakan prinsip kunci dalam arsitektur microservices untuk memastikan independensi dan decoupling.

6.1. auth_db (Auth Service)

- Tabel users: Menyimpan informasi dasar pengguna (ID, nama, email, password hash, detail OAuth jika ada).
- Tabel refresh_tokens: Menyimpan refresh token yang dikeluarkan, terkait dengan user_id, masa berlaku (expires_at), dan status pencabutan (revoked).

6.2. field_db (Field Service)

- Tabel fields: Menyimpan detail lapangan olahraga (ID, nama, tipe, lokasi, harga per jam, deskripsi, owner_id, status aktif).
- Tabel slots: Menyimpan slot waktu spesifik untuk setiap lapangan pada tanggal tertentu (ID, field_id, tanggal, waktu mulai, waktu selesai, status ketersediaan).
- Tabel field_owners: Menyimpan informasi pemilik lapangan (ID, user_id dari Auth Service, nama bisnis, rekening bank).

6.3. booking_db (Booking Service)

- Tabel bookings: Menyimpan detail booking (ID, user_id dari Auth Service, field_id dari Field Service, slot_id dari Field Service, status booking, jumlah DP, total pembayaran, tanggal booking, waktu, catatan).
- Tabel payments: Menyimpan riwayat pembayaran untuk setiap booking (ID, booking_id, tipe pembayaran (DP/lunas), jumlah, metode, status).
- Tabel reviews: Menyimpan ulasan pengguna untuk booking yang sudah selesai (ID, booking_id, rating, komentar).
- Tabel booking_slots: Tabel junction untuk mencatat slot yang dibooking (ID, booking_id, slot_id, tanggal, waktu).

7. Keamanan Sistem

Beberapa lapisan keamanan telah diimplementasikan untuk melindungi sistem:

1. Menggunakan JSON Web Tokens untuk memverifikasi identitas pengguna. Access Token berumur pendek untuk mengurangi risiko penyalahgunaan, dan Refresh Token digunakan untuk mendapatkan token baru.
2. Password pengguna tidak disimpan dalam bentuk plain text, melainkan di-hash menggunakan bcryptjs dengan 12 rounds salt. Ini melindungi password dari serangan brute-force atau kebocoran database.
3. Refresh token disimpan di database dan dapat dicabut (revoke) secara manual oleh pengguna (logout) atau secara otomatis oleh sistem. Ini memungkinkan kontrol atas sesi pengguna.
4. API Gateway menerapkan rate limiting (misalnya, 60 permintaan per menit per IP) untuk mencegah serangan DDoS atau brute-force pada endpoint publik seperti login dan register.
5. CORS (Cross-Origin Resource Sharing): Diaktifkan di setiap layanan dan Gateway untuk mengontrol domain mana yang diizinkan untuk mengakses API, mencegah serangan Cross-Site Request Forgery (CSRF) dan memastikan keamanan interaksi frontend-backend.
6. Meskipun tidak dijelaskan secara rinci di semua file, validasi input dasar dilakukan di controller (misalnya, authController.js memeriksa name, email, password tidak kosong dan panjang password). Ini mencegah serangan injeksi dan memastikan integritas data.
7. Menggunakan connection pooling dengan enableKeepAlive: true dan keepAliveInitialDelayMs: 0 untuk menjaga koneksi database tetap hidup dan responsif, mengurangi risiko connection timeout yang dapat dieksploitasi.
8. Setiap layanan memiliki penanganan kesalahan yang lebih baik, mengembalikan pesan kesalahan yang jelas tanpa mengekspor detail internal yang sensitif, dan mencatat kesalahan untuk debugging.